

Algoritmi e Strutture Dati – 02/02/2026

Esercizio A1 – Complessità – Punti ≥ 8

L'equazione di ricorrenza associata all'algoritmo è

$$T(n) = \begin{cases} 1 & n \leq 10 \\ 2T(n-10) + 1 & n \geq 11 \end{cases}$$

Utilizzando il Teorema delle Ricorrenze lineari di ordine costante, è possibile vedere che il costo della funzione è $T(n) = \Theta(2^n)$.

Errori ricorrenti Alcune soluzioni presentate dagli studenti contenevano questa ricorrenza:

$$T(n) = \begin{cases} 1 & n \leq 10 \\ 2T(n-10) + n & n \geq 11 \end{cases}$$

poi analizzata correttamente tramite il Teorema delle Ricorrenze lineari di ordine costante, dichiarando una complessità $T(n) = \Theta(2^n \cdot n)$. Dall'interpretazione delle descrizioni, mi sembra di capire che l'errore si annida nel considerare tutti i casi base "raggiunti" dalle chiamate ricorsive con costo $O(n)$; ma il caso base si verifica quando $n < 10$, che è un valore costante. Quindi il ciclo for contenuto nel caso base ha comunque costo costante.

Esercizio A2 – Albero sommato – Punti ≥ 10

L'algoritmo costruisce ricorsivamente un nuovo albero T' con la stessa struttura dell'albero di input T . Per ciascun nodo non nullo T :

- crea un nuovo nodo *new*,
- costruisce ricorsivamente i sottoalberi sommati dei figli sinistro e destro,
- inizializza *new.value* con *t.value* e poi vi aggiunge i valori contenuti nelle radici dei sottoalberi appena costruiti.

Poiché tali valori rappresentano le somme dei rispettivi sottoalberi, al termine *new.value* coincide con la somma dei valori di tutti i nodi del sottoalbero radicato in *t*. Nel caso base $t = \mathbf{nil}$, l'algoritmo restituisce $\mathbf{nil}$.

Ogni nodo dell'albero di input viene visitato esattamente una volta e in ciascun nodo svolgiamo un numero costante di operazioni. Pertanto il tempo di esecuzione è $\Theta(n)$, dove n è il numero di nodi di T .

```
TREE summedTree(TREE t)
```

```
if t == nil then
    return nil
else
    TREE new = Tree()
    new.left = summedTree(T.left)
    new.right = summedTree(T.right)
    new.value = T.value
    if new.left != nil then
        new.value += new.left.value
    if new.right != nil then
        new.value += new.right.value
    return new
```

Esercizio A3 – Massimo due – Punti ≥ 12

Prima soluzione ($\Theta(n^3)$). La soluzione più immediata consiste nell'esaminare tutti i sottovettori contigui $S[start \dots end]$ della stringa. Per ciascun sottovettore si costruisce l'insieme dei caratteri distinti che contiene; se tale insieme ha cardinalità al più 2, si aggiorna una variabile *maxSoFar* con la lunghezza

del sottovettore considerato. Poiché il numero di sottovettori è $\Theta(n^2)$ e per ciascuno di essi è necessario scansionare fino a n elementi per determinare i caratteri distinti, il tempo totale richiesto è $\Theta(n^3)$.

```

int maxTwo(item[] S, int n)
{
    int maxSoFar = 0
    for start = 0 to n - 1 do
        for end = start + 1 to n - 1 do
            SET chars = Set()
            for i = start to end do
                chars.insert(S[i])
            if chars.size() ≤ 2 then
                maxSoFar = max(maxSoFar, end - start + 1)
    return maxSoFar
}

```

Seconda soluzione ($O(n^2)$). Un miglioramento naturale consiste nel fissare l'indice di partenza *start* e nell'estendere progressivamente l'indice di fine *end*, mantenendo l'insieme dei caratteri distinti incontrati finora. Quando l'aggiunta di un nuovo carattere provoca la presenza di un terzo carattere distinto, il sottovettore non può più essere esteso e si passa al valore successivo di *start*. In questo modo si evita di ricostruire da zero l'insieme dei caratteri per ogni coppia (*start*, *end*). La complessità è $O(n^2)$ nel caso pessimo (tutti caratteri uguali); può scendere a $\Omega(n)$ nel caso di tre caratteri che si alternano, come per esempio nel caso della stringa "abcabcabc".

```

int maxTwo(item [] S, int n)
{
    int maxSoFar = 0
    for start = 0 to n - 1 do
        SET chars = Set()
        int end = start
        while chars.size ≤ 2 and end < n do
            chars.insert(S[end])
            if chars.size() ≤ 2 then
                maxSoFar = max(maxSoFar, end - start + 1)
            end++
    return maxSoFar
}

```

Terza soluzione ($O(n \log n)$). Il problema può essere risolto mediante la tecnica del divide-et-impera. La funzione `maxTwoRec(S, start, end)` restituisce la lunghezza massima di una sottostringa contigua di $S[start \dots end]$ che contenga al più due caratteri distinti.

Se l'intervallo ha lunghezza al più 2, la risposta è immediata – la lunghezza dell'intervallo. Altrimenti, si divide l'intervallo in due metà in corrispondenza di *mid* e si calcolano ricorsivamente le soluzioni ottimali nella metà sinistra e nella metà destra.

Resta da considerare il caso di una sottostringa che attraversa il taglio tra *mid* e *mid* + 1 (soluzione *cross*), e che quindi contiene necessariamente i caratteri $S[mid]$ e $S[mid + 1]$. In questo caso si mantiene un insieme *chars* dei caratteri distinti ammessi (al più due) e si calcola la massima estensione contigua attorno al centro. Per gestire correttamente anche il caso in cui $S[mid] = S[mid + 1]$, l'estensione viene calcolata due volte: una espandendo prima verso sinistra e poi verso destra, e una in ordine opposto. La migliore soluzione *cross* è il massimo tra i due risultati.

La risposta per l'intervallo corrente è quindi il massimo tra la soluzione migliore a sinistra, a destra e quella che attraversa il centro.

Il passo di combinazione richiede tempo lineare nella dimensione dell'intervallo, quindi

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

```

int maxTwo(item [] S, int n)
{
    return maxTwoRec(S, 0, n - 1)
}

```

```

int maxTwoRec(item [] S, int start, int end)
    if end - start + 1 ≤ 2 then
        return end - start + 1
    else
        int mid = ⌊(start + end)/2⌋
        int maxLeft = maxTwoRec(S, start, mid)
        int maxRight = maxTwoRec(S, mid + 1, end)

        SET chars = Set()
        chars.insert(S[mid])
        chars.insert(S[mid + 1])
        int i = goingLeft(S, start, mid - 1, chars)
        int j = goingRight(S, mid + 2, end, chars)
        int maxGoingLeftFirst = j - i + 1

        SET chars = Set()
        chars.insert(S[mid])
        chars.insert(S[mid + 1])
        int j = goingRight(S, mid + 2, end, chars)
        int i = goingLeft(S, start, mid - 1, chars)
        int maxGoingRightFirst = j - i + 1

        return max(maxLeft, maxRight, maxGoingLeftFirst, maxGoingRightFirst)

```

```

int goingRight(item [] S, int first, int end, SET chars)
    int j = first
    while j ≤ end and (chars.size() == 1 or chars.contains(S[j])) do
        chars.insert(S[j])
        j = j + 1
    return j - 1

```

```

int goingLeft(item [] S, int start, int first, SET chars)
    int i = first
    while i ≥ start and (chars.size() == 1 or chars.contains(S[i])) do
        chars.insert(S[i])
        i = i - 1
    return i + 1

```

Quarta soluzione ($O(n)$). È possibile risolvere il problema in tempo lineare tramite la tecnica della *finestra scorrevole* (*sliding window*). Manteniamo due indici *start* e *end* che delimitano una sottostringa $S[start \dots end]$ contenente al più due caratteri distinti. Estendiamo progressivamente *end* verso destra; quando l'aggiunta di $S[end]$ introduce un terzo carattere distinto, spostiamo *start* verso destra finché nella finestra non rimangono di nuovo al più due caratteri distinti. Durante l'esecuzione manteniamo l'insieme (o dizionario) dei caratteri presenti nella finestra e il loro numero di occorrenze, così da poter rimuovere un carattere quando il suo conteggio scende a zero. La variabile *maxSoFar* viene aggiornata con la massima lunghezza osservata.

```
int maxTwo(item [] S, int n)
```

```
int start = 0
int maxSoFar = 0
HASH count = Hash()                                % Dizionario: carattere → occorrenze nella finestra
for end = 0 to n - 1 do
    if count.lookup(S[end]) == nil then
        | count.insert(S[end], 1)
    else
        | count.insert(S[end], lookup(S[end]) + 1)
    while count.size() > 2 do
        | int c = count.lookup(S[start])
        | if c == 1 then
        | | count.remove(S[start])
        | else
        | | count.insert(S[start], c - 1)
        | start = start + 1
    maxSoFar = max(maxSoFar, end - start + 1)
return maxSoFar
```

Ogni indice *end* avanza da 0 a $n - 1$ una sola volta. Anche *start* avanza monotonamente e non supera mai n ; quindi ogni posizione viene rimossa dalla finestra al più una volta. Il tempo totale è $O(n)$.

Soluzioni errate Una soluzione errata ricorrente è quella di utilizzare un semplice insieme per considerare i caratteri diversi presenti, e cancellarlo quando si trova il terzo carattere diverso da quello precedente. Qualcosa del genere:

```
int maxTwo(item [] S, int n)
```

```
int maxSoFar = 0
int count = 0
SET letters = Set()                                % Insieme: caratteri diversi
for i = 0 to n - 1 do
    if letters.contains(S[i]) then
        | count = count + 1
        | maxSoFar = max(maxSoFar, count)
    else
        | if letters.size() == 2 then
        | | count = 0
        | | letters = Hash()
        | count = count + 1
        | maxSoFar = max(maxSoFar, count)
        | letters = insert(S[i])
return maxSoFar
```

Questo però non funziona con stringhe tipo "abccc", perché restituisce 3 (derivante dalle 3 "c") invece che 4 (derivante da "bccc"). Il reset elimina anche il secondo carattere che potrebbe ancora far parte della finestra ottima, quindi non mantiene una sliding window corretta.